Safety-Critical Embedded Software: A Performance and Safety Comparison of C and Rust RTOS Implementations

Simon Nenning
Department of Electronic Engineering & Entrepreneurship,
University of Applied Sciences Technikum Wien,
Vienna, Austria
es24m031@technikum-wien.at

***Abstract*—Real-time operating systems (RTOS) constitute the backbone of many embedded systems across domains such as automotive, aerospace, and industrial automation. Traditionally, these systems have been implemented in C due to its performance characteristics and low-level hardware control capabilities. However, such implementations frequently exhibit critical safety vulnerabilities arising from a lack of memory and concurrency safety guarantees. Rust, a modern systems programming language, has emerged as a promising alternative, providing compile-time assurances of safety without compromising performance.**

This research aims to empirically compare RTOS implementations in C and Rust, emphasizing safety, performance, and maintainability. Widely adopted C-based RTOS platforms, such as FreeRTOS and Zephyr, will be benchmarked alongside Rust-based alternatives like Tock OS and Rust bindings for FreeRTOS. The evaluation will be conducted on the STM32F407 (ARM Cortex-M4) and ESP32-C3 (RISC-V) microcontrollers. Performance metrics will be assessed using established benchmarking frameworks, such as Rhealstone and Thread-Metric, covering critical factors including task-switching latency, interrupt response time, memory footprint, and inter-process communication efficiency.

In addition to quantitative benchmarking, the study will evaluate code safety using static analysis tools and examine Rust's influence on development complexity and error prevention. The findings are anticipated to inform best practices for selecting programming models and RTOS platforms, particularly for future safety-critical embedded system designs.

***Keywords*—Real-time operating system (RTOS), embedded systems, Rust programming language, C programming language, safety-critical systems, memory safety, concurrency, static analysis, microcontroller, STM32F407, ESP32-C3, ARM Cortex-M4, RISC-V, FreeRTOS, Zephyr, Tock OS, performance benchmarking, interrupt latency, context switching, thread scheduling, system reliability, deterministic behavior, language safety, inter-process communication (IPC), embedded software engineering, low-level programming, embedded Rust, system performance, static memory analysis, RTOS comparison**

I. Introduction

Software is an integral component of embedded systems and must provide deterministic behavior with the shortest possible runtime. These requirements are particularly important given the typical application areas of embedded systems, such as the automotive, aerospace, and industrial automation sectors [1]. Traditionally, embedded software has been developed using programming languages such as C and C++, which meet performance demands but fall short regarding safety and security. This issue has become so prevalent that even the White House issued a statement advocating for a transition from C-based systems toward those using languages with stronger memory safety guarantees. [2] One of the most prominent and promising languages in this domain is Rust. This language provides valuable concepts to mitigate memory management issues while maintaining concise syntax and achieving performance levels comparable to C and C++ [3]. Rust eliminates entire classes of bugs without compromising runtime efficiency. Its design is particularly noteworthy, as it facilitates the development of reliable, low-level software, making it highly suitable for real-time and embedded applications [4].

These advantages are widely recognized within the community, prompting developers of real-time operating systems (RTOS) to adopt Rust. Some developers are creating entirely new kernels and operating systems using Rust. Notable examples include Tock OS [5] and Redox OS, which are pioneering secure and modular systems tailored specifically for microcontrollers. Meanwhile, established RTOS platforms such as FreeRTOS and Zephyr [6] are incorporating Rust through safe bindings and wrappers, enabling developers to write application software that safely interacts with the underlying operating system.

This trend extends beyond embedded systems. Major technology companies, including Google [7], Microsoft [8], and Amazon [9], have begun integrating Rust into their infrastructure to enhance security and performance.

Given this context, conducting a comparative study of RTOS implementations in C and Rust is timely and highly relevant. Such an analysis will offer insights into the trade-offs between traditional and modern approaches regarding system safety, performance, and maintainability. By evaluating metrics such as memory usage, execution time, and fault tolerance, this research aims to identify critical factors that influence the reliability of real-time systems implemented with Rust.

II. Contribution

The main contribution of this work is an empirical and comprehensive comparison between real-time operating systems implemented in C and those implemented in, or interfaced with, Rust. This study aims to evaluate the safety, maintainability, and performance of both languages in embedded environments, specifically focusing on systems deployable on STM32F407 (ARM Cortex-M4) and ESP32-C3 (RISC-V) microcontrollers.

This thesis contributes to the state of the art in the following ways:

- **Survey and Classification:** A detailed survey and classification of RTOSes that have active support for both C and Rust, including FreeRTOS, Zephyr, RIOT, NuttX, RT-Thread, and Tock OS. The focus will be on open-source and freely available systems.
- **Benchmarking Methodology:** The design and implementation of a reproducible benchmarking framework based on established RTOS benchmarking suites (e.g., Rhealstone [10], Thread-Metric [11], RTOSBench [12]), measuring critical performance metrics such as task switch latency, interrupt latency, memory footprint, and IPC overhead.
- **Safety Evaluation:** An analysis of safety guarantees that are offered by each RTOS through static analysis, memory access safety, and concurrency model comparison. Particular attention is given to how Rust's ownership model mitigates typical C errors such as race conditions, null pointer dereferences, and memory leaks.
- **Platform Evaluation:** Deployment and testing of selected RTOS configurations on STM32F407 and ESP32-C3 development boards to assess real-world applicability and hardware compatibility.
- **Practical Guidelines:** The formulation of best-practice recommendations for developers and system architects when choosing between C-based and Rust-based RTOS solutions in embedded safety-critical applications.

Through this contributions, this work aims to compare traditional embedded systems development in C and emerging ecosystems enabled by Rust. This should result in a guidance of future efforts in adopting safe systems programming for real-time applications

III. Relevance

The relevance of this work lies in the increasing demand for safe and efficient software for embedded real-time systems, as highlighted in the introduction. As embedded applications become more complex and critical across various industries, ensuring their reliability becomes paramount [13]. Real-time operating systems (RTOS) serve as a cornerstone for such systems; thus, their correctness significantly impacts overall system behavior and safety, necessitating meticulous attention and analysis [14].

Traditionally, RTOS kernels and applications have been implemented in C due to its low-level hardware access capabilities and predictable performance. However, C lacks built-in safety mechanisms, leaving systems vulnerable to numerous issues such as memory corruption, data races, and undefined behavior. Historically, these problems have resulted in severe security vulnerabilities and system failures, especially in safety-critical environments [15].

Currently, the most popular alternative to C in systems programming is Rust, as it provides memory and thread safety guaranteed at compile-time without sacrificing performance. Major industry players such as Meta [16], Dropbox [17], and Cloudflare [18] have already begun rewriting significant internal systems in Rust to enhance performance and security while reducing software defects. Rust is rapidly gaining traction in the embedded domain as well, evident through the emergence of various Rust bindings and wrappers for popular platforms. Notable examples include FreeRTOS [19] and Zephyr [6]. Additionally, developers are experimenting with fully Rust-based kernels, such as Tock OS [5], and even rewriting drivers for widely used platforms, like the Espressif IDF, entirely in Rust [20].

Despite the growing interest in using Rust for embedded systems, there remains a significant lack of direct comparisons between RTOS implementations in C and Rust. Existing studies typically explore Rust's applicability to low-level platforms; however, while these evaluations often provide detailed assessments of language-specific features or isolated performance metrics, they generally overlook explicit comparisons of memory safety models, scheduling behaviors, inter-process communication mechanisms, and hardware portability.

Sharma et al. explicitly highlight this research gap, noting that although Rust shows promise for safety-critical embedded systems, there remains insufficient empirical data comparing its use to traditional C-based RTOS platforms across multiple architectural targets and real-time constraints [21]. Consequently, developers and system architects currently lack the comprehensive insights required to make informed decisions regarding the adoption of Rust in real-time embedded systems.

By addressing this gap, this work aims to provide valuable insights into the pros, cons, and additional benefits that the Rust programming language offers for real-time embedded systems. It will help assess whether Rust's safety advantages translate effectively into practical embedded applications, and identify any potential performance or integration bottlenecks. These findings will be relevant not only to researchers but also to developers considering a transition toward safer embedded software development.

IV. Originality & Related Work

A. Originality

This thesis presents a novel analysis of real-time operating systems implemented in both C and Rust, with a focus on safety, performance, and maintainability metrics. The originality of this work lies in the empirical evaluation of RTOS implementations in both languages on identical hardware platforms—specifically, the STM32F407 (ARM Cortex-M4) and ESP32-C3 (RISC-V) microcontrollers. By deploying and benchmarking RTOSes such as FreeRTOS, Zephyr, and Tock

OS in C and Rust environments, this study aims to generate quantitative data on performance metrics including task switch latency, interrupt handling, and memory footprint. Additionally, the thesis will evaluate the safety features and developer experience provided by each language.

B. Related Work

Several studies have explored the use of Rust in embedded systems. Sharma et al. discussed the current challenges and progress in adopting Rust for embedded development, citing memory safety and language support as key areas of concern [21].

Levy et al. demonstrated the feasibility of building a complete embedded stack in Rust, highlighting the safety advantages of Rust's type system and ownership model in kernel-level programming [4].

Furthermore, Plauska et al. compared the runtime performance of various programming languages, including Rust and C++, on the ESP32, concluding that Rust offers competitive performance while significantly enhancing memory safety [22].

Despite these contributions, a gap remains in systematic comparisons of RTOS implementations in C and Rust on identical microcontroller platforms. This research aims to fill that gap by providing side-by-side benchmarks and analyses, focusing not only on performance but also on maintainability and system safety.

V. Methodology & Evaluation

A. Approach

The research will follow an empirical methodology to compare real-time operating systems (RTOS) implemented in C and Rust across several dimensions: performance, safety, and maintainability. The systems under test will include widely used RTOS platforms such as FreeRTOS and Zephyr (C implementations), and their Rust-compatible counterparts, either through official bindings (e.g., `freertos-rust` [19]) or native Rust implementations (e.g., Tock OS [5]).

The experimental setup will target two representative microcontroller architectures:

- **STM32F407** (ARM Cortex-M4) [23]
- **ESP32-C3** (single-core RISC-V) [24]

The workflow will consist of the following phases:

1) **Platform Preparation:** Set up hardware and toolchains (e.g., STM32CubeIDE, ESP-IDF, Rust Embedded HAL).
2) **RTOS Configuration:** Deploy the selected RTOSes on each target, with equivalent scheduling policies, stack sizes, and configurations where possible.
3) **Benchmark Suite Implementation:** Implement and run established RTOS benchmarks such as Thread-Metric and Rhealstone [4], [22] to measure:
   - Context switch latency
   - Interrupt handling latency
   - Memory footprint (RAM/Flash usage)
   - Inter-process communication (IPC) throughput and latency
   - Scheduler overhead
   - Power consumption under idle and load
4) **Safety and Maintainability Analysis:** Evaluate code safety using static analysis tools (e.g., `Clang Analyzer`, `MIRAI` for Rust), check for memory safety violations, and assess code complexity and modularity.

All tests will be automated to ensure consistency and repeatability across runs and platforms.

B. Evaluation and Validation

To validate the results and ensure objectivity:

- Each test will be executed a minimum of 1000 times to average out timing jitter and hardware variances.
- GPIO-based instrumentation and high-resolution timers will be used for accurate timing measurements.
- Results will be statistically analyzed to report mean, variance, and worst-case latencies for each metric.
- Measurements will be compared against documented baseline performance (where available) from existing literature [21], [22].
- The correctness of Rust-based implementations will be verified via unit tests and memory safety assertions provided by the Rust compiler.
- All benchmark code and data will be made available in a public repository to ensure reproducibility.

By combining quantitative benchmarking with static analysis and reproducibility, this methodology ensures a robust and transparent evaluation of the practical benefits and trade-offs of using Rust versus C in embedded RTOS development.

References

[1] S. Susnjara and I. Smalley, "What is a real-time operating system (rtos)?" https://www.ibm.com/topics/real-time-operating-system, 2023, accessed: 2025-05-18.

[2] O. of the National Cyber Director, "Back to the building blocks: A path toward secure and measurable software," February 2024, accessed: 2025-05-09. [Online]. Available: https://www.whitehouse.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf

[3] W. Bugden and A. Alahmar, "Rust: The programming language for safety and performance," *arXiv preprint arXiv:2206.05503*, 2022. [Online]. Available: https://arxiv.org/abs/2206.05503

[4] A. Levy, B. Campbell, and P. Levis, "Multiprogramming a 64kb computer safely and efficiently," in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017, pp. 234–251.

[5] T. T. O. Project, "Tock: A secure embedded operating system for microcontrollers," https://www.tockos.org, 2023, accessed: 2025-05-18.

[6] T. Z. Project, "Rust support in zephyr rtos," https://docs.zephyrproject.org/latest/develop/languages/rust/index.html, 2024, accessed: 2025-05-18.

[7] Google Security Blog, "Rust in the android platform," https://security.googleblog.com/2021/04/rust-in-android-platform.html, 2021, accessed: 2025-05-18.

[8] R. Levick, "Why rust for safe systems programming," https://msrc.microsoft.com/blog/2019/07/why-rust-for-safe-systems-programming/, 2019, accessed: 2025-05-19.

[9] AWS Open Source Blog, "Announcing the general availability of bottlerocket: An open source linux distribution purpose-built to run containers," https://aws.amazon.com/blogs/opensource/announcing-the-general-availability-of-bottlerocket-an-open-source-linux-distribution-purpose-built-to-run-containers/, 2020, accessed: 2025-05-19.

[10] R. P. Kar, "Implementing the rhealstone real-time benchmark," *Dr. Dobb's Journal*, pp. 46–102, 1989.
[11] J. Ganssle, "Measure your rtos's real-time performance," 2005, accessed: 2025-05-19. [Online]. Available: https://www.embedded.com/measure-your-rtoss-real-time-performance/
[12] M. P. E. GmbH, "Rtosbench: A benchmark suite for real-time operating systems," https://github.com/methodpark/zephyr-rtosbench, accessed: 2025-05-18.
[13] M. Technologies, "Role of embedded system and its future in industrial automation," https://moschip.com/blog/semiconductor/role-of-embedded-system-and-its-future-in-industrial-automation/, 2022, accessed: 2025-05-18.
[14] W. River, "What are real-time and safety-critical systems?" https://www.windriver.com/solutions/learning/real-time-safety-critical-systems, 2025, accessed: 2025-05-18.
[15] D. of Defense, "The case for memory safe roadmaps," https://media.defense.gov/2023/Dec/06/2003352724/-1/-1/0/THE-CASE-FOR-MEMORY-SAFE-ROADMAPS-TLP-CLEAR.PDF, 2023, accessed: 2025-05-18.
[16] E. Garcia, "Programming languages endorsed for server-side use at meta," https://engineering.fb.com/2022/07/27/developer-tools/programming-languages-endorsed-for-server-side-use-at-meta/, 2022, accessed: 2025-05-18.
[17] C. Metz, "The epic story of dropbox's exodus from the amazon cloud empire," https://www.wired.com/2016/03/epic-story-dropboxs-exodus-amazon-cloud-empire/, 2016, accessed: 2025-05-18.
[18] Cloudflare, "Open sourcing pingora: our rust framework for building http proxy services," https://blog.cloudflare.com/pingora-open-source/, 2023, accessed: 2025-05-18.
[19] Lobaro, "Freertos rust," https://github.com/lobaro/FreeRTOS-rust, 2025, accessed: 2025-05-19.
[20] Frostie314159 and J. Devreker, "Liberating wi-fi on the esp32," Chaos Communication Congress (38C3), 2023, accessed: 2025-05-19. [Online]. Available: https://media.ccc.de/v/38c3-liberating-wi-fi-on-the-esp32
[21] A. Sharma, S. Sharma, S. Torres-Arias, and A. Machiry, "Rust for embedded systems: Current state and open problems," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 1–15.
[22] I. Plauska, A. Liutkevičius, and A. Janavičiūtė, "Performance evaluation of c/c++, micropython, rust and tinygo programming languages on esp32 microcontroller," *Electronics*, vol. 12, no. 1, pp. 1–15, 2023.
[23] STMicroelectronics, "STM32F405xx/STM32F407xx Datasheet," https://www.st.com/resource/en/datasheet/dm00037051.pdf, 2023, accessed: 2025-05-19.
[24] Espressif Systems, "ESP32-C3 Series Datasheet," https://www.espressif.com/sites/default/files/documentation/esp32-c3$_d atasheet_e n.pdf$, 2020, $accessed: 2025-05-19$.